

Aufgabenblatt: ArrayList

Aufgabe 1: Summe einer ArrayList

Beschreibung: Eine `ArrayList` ist die dynamische Alternative zum `Array`: Sie wächst beim Hinzufügen automatisch und schrumpft beim Entfernen. Mit einer `For-each`-Schleife lassen sich alle Elemente genau wie bei einem `Array` durchlaufen.

Aufgabenstellung: Erstelle eine `ArrayList<Double>` und füge mindestens fünf Zahlenwerte direkt im Code hinzu. Berechne die Summe aller Elemente mit einer Schleife und gib das Ergebnis aus.

Die Summe aller Zahlen beträgt: 24.5

Aufgabe 2: Highscore-Liste

Beschreibung: `Collections.sort()` sortiert eine `ArrayList` aufsteigend. Mit `Comparator.reverseOrder()` als zweitem Argument wird absteigend sortiert. `.indexOf()` findet die Position eines Werts in der sortierten Liste, `.set()` ersetzt ein Element an dieser Position durch einen neuen Wert.

Aufgabenstellung: Lege folgende `ArrayList<Integer>` direkt im Code an:

```
ArrayList<Integer> scores = new ArrayList<>();
scores.add(850);
scores.add(1200);
scores.add(430);
scores.add(975);
scores.add(1500);
scores.add(620);
```

Gib die Liste unsortiert aus. Sortiere sie mit `Collections.sort()` aufsteigend und gib sie aus. Sortiere sie anschliessend absteigend und gib sie erneut aus. Ermittle mit `.indexOf()` den Index des Werts 850 in der absteigend sortierten Liste und ersetze ihn mit `.set()` durch 900.

```
Original:    [850, 1200, 430, 975, 1500, 620]
Aufsteigend: [430, 620, 850, 975, 1200, 1500]
Absteigend:  [1500, 1200, 975, 850, 620, 430]
Index von 850 (absteigend): 3
Nach set():  [1500, 1200, 975, 900, 620, 430]
```

Aufgabe 3: Einzigartige Namen

Beschreibung: `.contains()` prüft, ob ein Wert bereits in der Liste vorhanden ist. In Kombination mit einer `while`-Schleife und einem Sentinel-Wert lässt sich eine Eingabe-Routine aufbauen, die Duplikate aktiv verhindert.

Aufgabenstellung: Lese mit einer `while`-Schleife so lange Namen ein, bis der Benutzer "stop" eingibt. Prüfe vor dem Hinzufügen mit `.contains()`, ob der Name bereits vorhanden ist. Ist er es, erscheint eine Meldung. Am Ende wird die vollständige Liste ausgegeben.

```
Gib einen Namen ein (stop zum Beenden): Anna
Gib einen Namen ein (stop zum Beenden): Ben
Gib einen Namen ein (stop zum Beenden): Anna
Dieser Name ist bereits vorhanden.
Gib einen Namen ein (stop zum Beenden): stop
Liste aller Namen: [Anna, Ben]
```

Aufgabe 4: Warenkorb

Beschreibung: Eine `ArrayList` eignet sich ideal für Menü-gesteuerte Programme: Elemente können jederzeit hinzugefügt und entfernt werden, ohne die Größe vorab festzulegen. `.contains()` stellt sicher, dass beim Entfernen zuerst geprüft wird, ob das Element überhaupt vorhanden ist.

Aufgabenstellung: Implementiere einen Warenkorb mit einer `ArrayList<String>` und folgendem Menü. Das Programm läuft in einer `while`-Schleife, bis der Benutzer Option 4 wählt.

```
Wähle eine Option:
1 - Produkt hinzufügen
2 - Produkt entfernen
3 - Warenkorb anzeigen
4 - Beenden
Deine Wahl: 1
Welches Produkt möchtest du hinzufügen? Milch
Milch wurde hinzugefügt.
```

[Bonus] Playlist verwalten

Beschreibung: `.add(index, element)` fügt ein Element an einer bestimmten Position ein und verschiebt alle folgenden Elemente nach hinten. In Kombination mit `.remove(index)` lässt sich ein Element aus seiner aktuellen Position herauslösen und eine Position früher einfügen.

Aufgabenstellung: Implementiere eine Playlist mit einer `ArrayList<String>` und folgendem Menü. Das Programm läuft bis der Benutzer Option 5 wählt. Beim Verschieben nach oben tauscht ein Song seinen Platz mit dem davor befindlichen Song. Ist der Song bereits an erster Stelle, erscheint eine Meldung.

=== Meine Playlist ===

- 1 - Song hinzufuegen
- 2 - Song entfernen
- 3 - Song nach oben verschieben
- 4 - Playlist anzeigen
- 5 - Beenden

Auswahl: 4

- 1. Bohemian Rhapsody
- 2. Stairway to Heaven
- 3. Hotel California

Auswahl: 3

Song nach oben verschieben: Hotel California

'Hotel California' ist jetzt auf Platz 2.

Auswahl: 4

- 1. Bohemian Rhapsody
- 2. Hotel California
- 3. Stairway to Heaven